Online Training
C# 6.NET Online Training Program
C# .NET Core Training
Microservices Online Training
Dotnet Online Training
Microsoft Azure Training
Angular Training

Introduction & Environment Setup
Setup
First Computer Works
Introduction to Programming Languages
How Computer Programs Work
Different Types of Applications
Programming Methodologies
Algorithms Flowcharts & Pseudocode
Introduction to .NET Framework
.NET Framework Architecture and Components
Introduction to C# Programming Language
How to Download and Install Visual Studio 2015 Version
Creating First Console Application using Notepad
.NET Developer Roadmap for 2021
Coding Standard Best Practices

C# .NET Basics
Basic Parts of Computer
Program
Methods and Properties of Console Class in C#
Data Types in C#
Variables in C#
Constants in C#
Variables in C#
Comment Rules
Comment Rule Statements in C#
Operators in C#
Arithmetic Operators in C#
Switch Statements in C#
Loops in C#
While Loop in C#
Do While Loop in C#
For Loop in C#
Nested Loop in C#
Continue Statement in C#
Goto Statement in C#
Functions in C#
Overrides Equals Method in C#
Operator Overloading in C#
Difference Between Content Filtering and Data Filtering
Checked and unchecked keywords in C#
Ternary and Nullable Operators in .NET
Boxing and Unboxing in C#

Exception Handling
Exception Handling in C#
Multiple Catch Blocks in C#
Finally Block in C#
How to Create Custom Exception in C#
Error Description in C#
Exception Handling Abuse in C#

Events, Delegates and Lambda Expressions in C#
Course Structure of Events, Delegates and Lambda Expressions
Introduction to Events
Rules of Events, Delegates and Event Handler in C#
Anonymous Method in C#
Lambda Expressions in C#
Events in C# with Examples

Array Handling
Multidimensional Array in C#
Jagged Array in C#
How to Pass Array Object Function in Type Safe Manner in C#
How to Retrieve Data from a Function Using Out Parameter in C#
How to Retrieve Data from Property of Thread Class in C#
Thread Synchronization in C#
Lock in C#
Monitor Class in C#
Semaphore Class in C#
SemaphoreSlim Class in C#
Deadlock in C#
Performance Testing of a Multi-threaded Application
Thread Pool in C#
Foreground and Background Threads in C#
AutoResetEvent and ManualResetEvent in C#
Thread.Join in C#
Thread.Priority in C#
How to Terminate a Thread in C#
Timer Thread Communication in C#
How to Debug a Multi-threaded Application in C#

Collections in C#
Array in C#
ICI Array in C#
ICI Hashtable and Dictionary in C#
Collections and Generics in C#
Arraylist in C#
Hashtable in C#
Non Generic Stack in C#
Non Generic Queue in C#
Non Generic Collections in C#
Advantages and Disadvantages of Non Generic Collections in C#
Generic Collections in C#
Generic Stack in C#
Generic Queue in C#
Generic Collections in C#
Generic List Collections in C#
How to Sort a List of Complex Type in C#
Generic Dictionary in C#
Dictionary Collection Class in C#
Generic Sorted Dictionary Collection Class in C#
Generic Sorted List Collection Class in C#
Generic SortedSet Collection Class in C#
Generic LinkedList Collection Class in C#
Generic Queue Collection Class in C#
Concurrent Dictionary Collection Class in C#
Concurrent Queue Collection Class in C#
Concurrent Stack Collection Class in C#
Concurrent Bag Collection Class in C#
Blocking Collection in C#

File Handling
File Handling in C#
FileStream Class in C#
StreamReader and StreamWriter in C#
File Class in C#
TextReader and TextWriter in C#
StringReader and StringWriter in C#
File Operations in C#
Directory Class in C#
Directory Operations in C#
Asynchronous File Handling in C#
Asynchronous Streams in C#

Parallel Programming
Task Parallel Library in C#
Parallel For in C#
Parallel Foreach Loop in C#
Parallel Invoke in C#
Parallel Extensions Cancellation Framework in C#
How to Cancel Parallel Operations in C#
Async Methods Tasks Async and Wait Keywords in C#
Interlocked.Add in C#
Parallel.Invoke in C#
Multithreading in Windows Forms Application in Parallel Programming in C#

AutoHotkeys
AutoHotkeys in C#
AutoHotkeys Complex Mapping in C#
How to Map Complex Type to Simple Type in C#
AutoHotkeys Ignore Remap in C#
Fixed and Dynamic Values in Destination Property in AutoHotkeys

Optional Parameter, Indexers and Enum
How to Make Optional Parameters in C#
Indexers in C#
Indexers Beat Time Example in C#
Enums in C#

.NET Framework Architecture
DOT.NET Framework
Common Language Runtime in .NET Framework
.NET Program Execution Process
Intermediate Language (IL) in C# & ASP.NET
Common Type System in .NET Framework
Managed and Unmanaged Code in .NET Framework
Assembly DLL EXE in .NET Framework
Reflection in .NET
DLL Hell Problem and Solutions in .NET Framework

Var, Dynamic and Reflection
Reflection in C#
Dynamic Type in C#
Var Keyword in C#
Var vs Dynamic in C#
Dynamic vs Reflection in C#
Anonymous Types in C#
Var vs Var in C#
Named Parameters in C#

C# 7.0 new Features
C# 7.0 New Features
Enhancement in Out Variables in C# 7.0
Pattern Matching in C#
Digit Separators in C# 7
Tuples in C# 7
Splitting Tuples in C# 7
Local Functions in C# 7
Ref Returns and Ref Locals in C# 7
Generalized Async Return Types in C# 7
Throw Expressions in C# 7
Throw Expression in C#
Async Main in C#

C# 8 New Features
C# 8 New Features
Readonly Structs in C#
Default Interface Methods in C#
Pattern Matching in C#
Using Declarations in C#
Static Local Functions in C#
Disposable Ref Structs in C#
Nullable Reference Types in C#
Asynchronous Streams in C#
Asynchronous Disposable in C#
Indices and Ranges in C#
Null Coalescing Assignment
Operator in C#
Unmanaged Constructed Types in C#
Stackalloc in C#

Most Popular C# Books
Most Recommended C# Books
Most Recommended Data Structure and Algorithm Books

Intermediate Language (IL)DASM & (IL)ASM) Code in C#

Back to C.NET Tutorials For Beginners and Professionals

Intermediate Language (IL)DASM & (IL)ASM) Code in C#.NET

In this article, I will discuss the **Intermediate Language (IL)DASM & (IL)ASM) Code** in C#.NET with Examples. Please read our previous article where we discussed the .NET Program Execution Process in detail. **(IL)DASM** stands for Intermediate Language Disassembler and **(IL)ASM** stands for Intermediate Language Assembler. As part of this article, we will discuss the following points, and at the end of the article, you will understand all about Intermediate Language (IL) Codes in C#.

- **What happens when we compile a .NET Application?**
- **Understanding the Intermediate Language (IL) Code in C#?**
- **What are (IL)DASM and (IL)ASM?**
- **How do you view the Intermediate Language code in C#?**
- **What is Manifest?**
- **How do you disassemble the Intermediate Language code to a text file?**
- **How do you rebuild an assembly from a text file that contains manifest and IL?**

What happens when we compile a .NET Application?

When we compile any .NET application, it will generate an assembly with the extension of either a DLL or an EXE. For example, if you compile a Windows or Console application, then you will get an EXE; whereas if you compile a Web or Class library project, then you will get a DLL. Irrespective of whether it is DLL or EXE, an assembly consists of two things: **Manifest and Intermediate language**. Let us understand what the Intermediate Language and Manifest look like in .NET Framework with an example.

Understanding Intermediate Language (IL)DASM & (IL)ASM) Code in C#.NET

In order to understand Intermediate Language Code (IL)DASM and (IL)ASM) in C#, let us create a simple console application. Once you create the console application, please modify the Program.cs class as shown below.

```
namespace ILDASMExample
{
    class Program
    {
        static void Main(string[] Args)
        {
            Console.WriteLine("Understanding ILDASM and ILASM");
            Console.Read();
        }
    }
}
```

Now, build the application. Once you build the application, the console source code is compiled and intermediate language code is generated and stored in an assembly. In order to see the assembly, go to the **bin\Debug** folder in the Project and select **Open Folder in File Explorer** option and then go to the **bin** or **Debug** folder and you should see an assembly with .exe extension generated as shown in the below image as it is a console application.

How to view the Intermediate Language Code in Manifest?

The .NET Framework provides a tool called **ILDASM (Intermediate Language Disassembler)** to view the code of the intermediate language in .NET Framework. In order to use the ILDASM tool, you need to follow the below steps. Open Visual Studio Command Prompt in Administrator mode as shown in the below image.

Once you open the Visual Studio command prompt in administrative mode, then type the **ildasm.exe C:\ILDasmExample\ILDASMExample\bin\Debug\ILDASMExample.exe** command. Here, it is provide the exact path where your exe is generated. My EXE is generated in the path **D:\ILDASMExample\ILDASMExample\bin\Debug\ILDASMExample.exe**, so I execute the following command in the command prompt.

```
Administrator: Developer Command Prompt for VS2015
D:\ILDASMExample>ildasm.exe D:\ILDASMExample\ILDASMExample\bin\Debug\ILDASMExample.exe
```

Once you type the above command and press enter, it should open the following ILDASM window.

As you can see, the assembly consists of two things: **Manifest and Intermediate language**. Let us first discuss the intermediate language code, and then we will discuss what Manifest is. Now, let us expand the ILDASMExample and compose it with our code. For a quick navigation purpose, please have a look at the below image. There is a constructor in ILDASM and this is because, by default, the .NET Framework provides a default constructor when there is no constructor in your class. You can also see the Main method in the intermediate language code.

ILDASMExample.exe
Class name
Class Program
Method name
Main Method
Intermediate Language
Intermediate Language Code
Method Signature
Method Body
Method Name
Method Signature
Method Body
Method Name
Method Signature
Method Body
Method Name